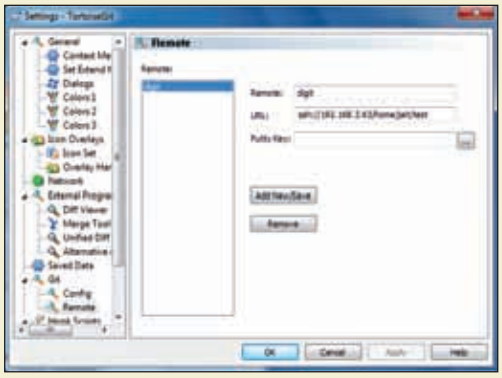
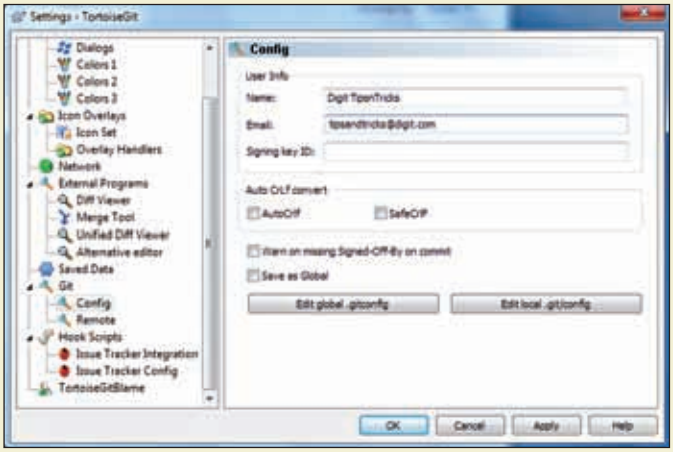


Configuring remote machine URL and name

On the other side repeat the process from the beginning, but keep the folder empty. So that your data from the other end can be pushed here. To push the data from one end, open the context menu in the repo folder and select "Push..." from the TortoiseGit submenu. In the dialog that follows select remote, and click on the manage button to specify the remote connection. Specify the name (this should be meaning full something like Home or Office) for the connection and specify the URL for the remote machine.Before it starts copying it will ask you to login to the remote machine, enter your username and password (This will require you to setup an OpenSSH server at both the ends, you can get one for Windows called MobaSSH from here http://bit.ly/ykZyeJ). And thats all there s to it, all your files will be added to the remote location. When you make changes there the files can be pushed from that location to this repository. So that all your updates on that machine are reflected here.

If you're afraid of Google

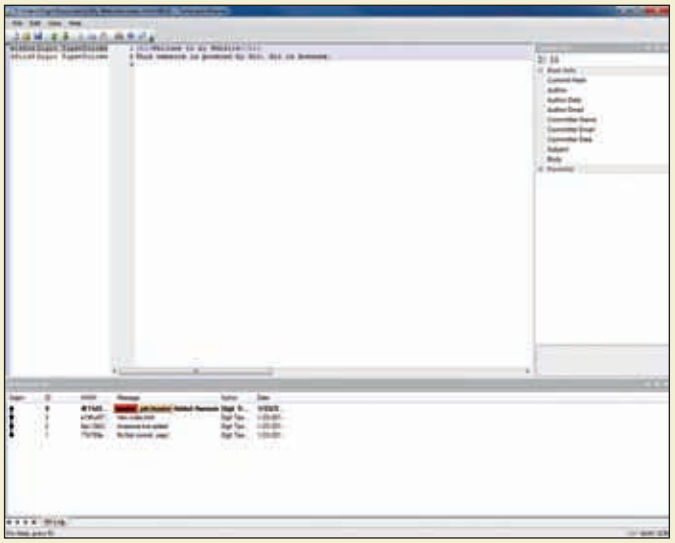
Google Docs is awesome with its auto-save, versioning and concurrent editing. In case you don't want to leave the cosy confines of your desktop office package (read MS Office) or you just don't trust the search engine giant with your precious data, but still want versioning, document tracking then Git comes to your rescue. Have your colleagues pull the working directory to their local branches, and at the end of the day you can just merge all the branches back to the master to make all the documents reflect the latest changes. In case someone messes something up you can always track the changes made and find out which changes were made by whom, and you can revert to the pre-merge situation by resetting the branch to a particular commit. The advantages in working this way are many, you wield a firmer control on versioning and tracking document changes.



Configuring Git

There is a caveat here though, Git treats Word, Excel and PowerPoint files as binary files and running `diff`, to determine the changes will be difficult. You can force git to actually find the difference between two Word files by editing the `.gitattributes` file. To force git to find the difference between two Word files add the following line to .gitattributes

```
*.doc diff=word
```



Lets play the blame game!

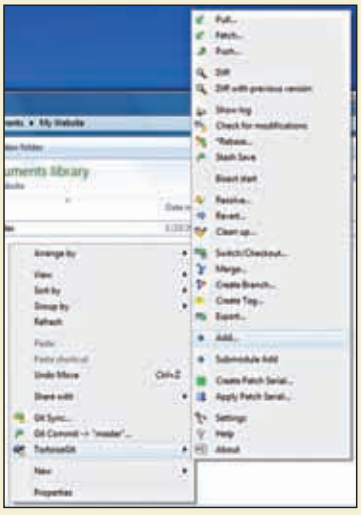
You could even save your Word files as a Word XML file and be able to determine the changes without editing the configuration file. There are no such facilities for Excel and Powerpoint, but you can use a comparison tool like WinMerge (http://bit.ly/x3mW5y) with Office plugin (http://bit.ly/wDyRUT) to manually find the changes made in the file.

Whodunnit?

All this talk about many people working on the same file (albeit on their own copies, the changes are merged to one branch) may make you wonder about how you can track the changes that the files undergo through their lifetime. Its infact extremely simple, select the file that you want to track, right click and select Blame from the TortoiseGit submenu. This will open a dialog,listing all the changes made in the last version and by whom. There is also a Git Revision List which will show when and by whom the changes were made.

Going back in time

To go back to a previous commit, inside the repo folder right-click and select `Show Log` from the TortoiseGit menu. After that you can right click previous commits and reset the branch to that commit. This will make the content same as what it was during that particular commit.



Git context menu